SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Ashira Bridget Susan S	**Reg. No.:**	192524316
Date:	15-10-2026	**Duration:**	60 minutes

Code of Conduct

*I, **Ashira Bridget Susan S (192524316),** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
    - Explain the problem in your own words.
    - Identify the objectives of the proposed system.
    - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
    - Identify all the stakeholders involved in the system.
    - Explain the roles and responsibilities of each stakeholder.
    - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
    - Describe the existing system or current process.
    - Identify its strengths and limitations.
    - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
    - Design a software-based solution for the given problem.
    - Describe the major functional modules of the proposed system.
    - Identify the functional and non-functional requirements.
    - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
    - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
    - Justify why the selected methodology/framework is suitable for the proposed system.
    - Explain how it supports successful project development and maintenance.
6. **Discuss Expected Outcomes and Limitations**
    - Describe the expected benefits of the proposed solution.
    - Explain how the solution addresses the identified challenges.
    - Discuss potential risks, implementation challenges, and limitations.
    - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology / Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Annexure A – Case Study Analysis
Autonomous Heavy-Duty Truck Platooning Control & Telemetry Synchronization Framework

1. Understand the Problem Statement

For instance, I consider cyclists in a race: the rider at the back tucks in behind the one in front to dodge the wind and save energy. Truck platooning is the same trick with 40-tonne trucks. A lead truck sets the speed and a few trucks follow about 10 metres behind it, so the air drag on every truck drops and fuel is saved. The catch is the speed. At 100 km/h a truck covers about 27.8 m every second, so a 10 m gap is only around 0.36 seconds. A human driver needs roughly 1.5 seconds just to react, and by then the truck has already moved about 40 m. Only computer software can safely maintain this close distance by sharing instant driving data between trucks faster than a human driver. The problem is therefore to build a system that keeps a small convoy tightly spaced, keeps all the trucks "talking" in real time, and still behaves safely when the wireless link fails.

Objectives of the proposed system:

- Keep the control loops of all trucks in sync over ad-hoc wireless links, targeting a 2 ms timing budget.
- Use a string-stable control algorithm so that a braking wave does not grow as it passes down the line.
- Coordinate safe lane changes, merging, platoon forming and platoon splitting on the highway.
- Detect a communication dropout immediately and move to a safe, decoupled separation.
- Keep a tamper-proof "black box" log across all trucks in the platoon.

Scope: The project covers small convoys (about 3 to 5 trucks) of semi-autonomous trucks on highways, which means the on-board control software, the truck-to-truck communication layer, the maneuver logic, the fault handling and the shared logging. It does not cover building the trucks' hardware, fully driverless operation in cities, or mixed traffic on ordinary roads. A driver stays on board every truck as a fallback.

2. Identify Key Stakeholders and Challenges

Quite a few people are affected by a convoy of trucks, not just the drivers. The table shows who they are, what they do, and what makes their life difficult.

Stakeholder	Role and responsibility	Main challenge
Lead truck driver	Sets speed and route, supervises the platoon, decides when to form or split it.	Carries responsibility for every truck behind and cannot brake carelessly.
Follower truck drivers	Monitor the system and take over when an alert comes.	Staying alert while the truck drives itself.
Fleet operator	Plans convoys, pays for the system, owns the data.	Balancing fuel savings against equipment cost and mixed truck brands.
Software and engineering team	Designs, builds, tests and updates the controllers.	Safety-critical code with very tight timing.
Network providers	Provide the wireless, cellular and satellite links.	Coverage gaps, delay and jitter.
Highway authorities	Approve platooning and set rules for gaps and lanes.	No common standard, tough certification.
Other road users	Share the highway with the convoy.	Cars cutting into a 10 m gap; fear of long truck groups.
Insurers and investigators	Judge what happened after an incident.	Need logs that cannot be edited.

Major challenges: The biggest one is timing: a late packet at 100 km/h can mean a crash. Wireless links are never perfect and can drop out in tunnels or weak-signal areas. Small errors can also add up along the line, so the last truck brakes much harder than the first (the “slinky effect”). Other challenges are cars squeezing into the gap, trust and liability if something goes wrong, and the lack of standards between truck makers.

3. Analyze the Current Approach

How things work today: Right now every long-haul truck is driven by its own driver. Many use adaptive cruise control (ACC), which uses a front radar and keeps a gap of about 1.5 to 2 seconds, i.e. roughly 40 to 55 m at 100 km/h. Trucks do not share their braking or throttle state with each other. Lane changes are done by the drivers, and records come from separate tachographs or event recorders in each truck.

Strengths: It is a proven, familiar and legally accepted setup. It does not depend on a wireless link, drivers use their own judgement in odd situations, and it is cheaper to run than a new system.

Limitations: The gap is so big that most of the drag saving is lost, since studies show the benefit mainly appears at much shorter gaps. A follower finds out about braking only after the radar sees the gap shrinking, so each truck reacts a little later than the one in front and the delay builds up down the line. Driver fatigue and the driver shortage remain, and every truck keeps its own separate log that can be altered or may be incomplete.

Gaps to be addressed: There is no shared real-time state between trucks, no guarantee of string stability, no proper procedure for forming, splitting and lane changes as a group, no tamper-proof shared log, and no graceful fallback when communication is lost.

4. Propose a Suitable Solution Using Software Engineering Principles

Overall idea: This software system is installed on every truck and is built in layers. Sensor data goes into a telemetry and time-sync layer, a controller turns it into brake, throttle and steering commands, and the trucks keep exchanging data and "heartbeat" messages over the wireless link. Every truck also stores a copy of a shared black-box log.

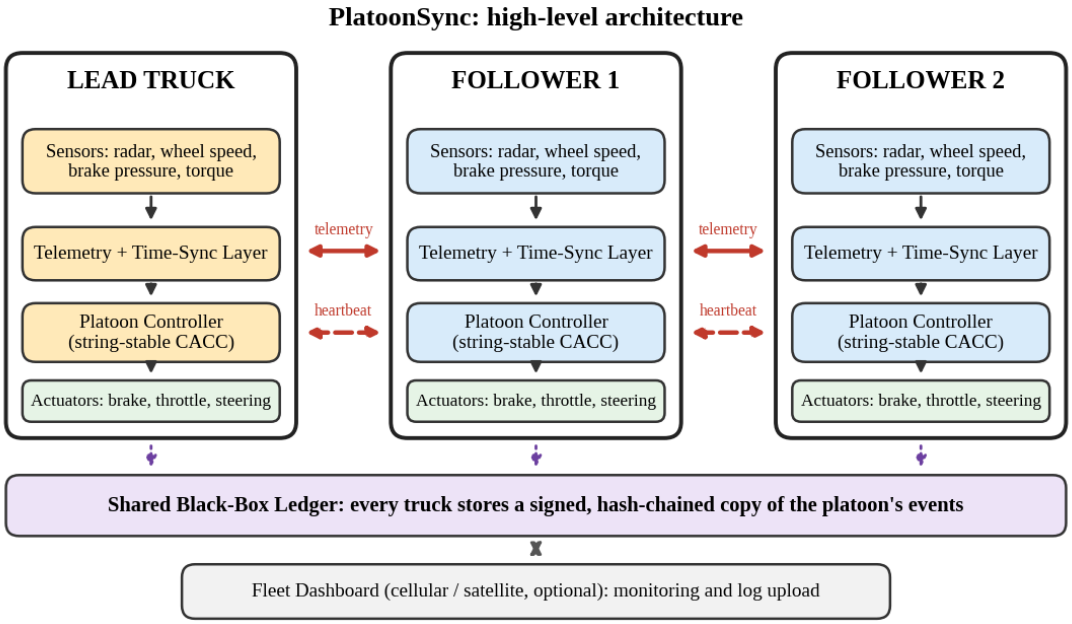


Figure 1: High-level architecture of the software

Major functional modules:

Module	What it does
Telemetry Acquisition	Reads radar distance, wheel speed, brake pressure and engine torque at high frequency and timestamps every sample.
Time-Sync and Comms Layer	Keeps all truck clocks aligned (GPS time or PTP) and sends compact, prioritised packets over the V2V link.
Platoon Controller	Computes throttle and brake commands to hold the gap; uses the lead truck's data as feed-forward.
Maneuver Manager	A state machine for Forming, Cruising, Lane Change, Merging and Splitting, with an acknowledgement from every truck at each step.
Fault Detection and Safe Decoupling	Watches heartbeats and sensor health, and moves the platoon to a safe fallback mode.

Module	What it does
Black-Box Logger	Records events in an append-only, hash-chained and signed log, copied to all trucks.
Fleet Dashboard	Lets the operator view platoon status and collect logs when a cellular or satellite link is available.

How the key objectives are met: For string stability, each follower works on its spacing error (actual gap minus a desired gap based on a small time headway) with a PD-style law, and adds the lead truck's and the predecessor's acceleration as feed-forward. The design is tuned so a disturbance does not grow from truck to truck. For the 2 ms target, every packet carries a timestamp, the local controller runs at a fixed high rate (about 500 Hz), and a simple predictor fills in if a packet arrives late, so the loop never sits waiting on the radio. For dropouts, if about three heartbeats in a row are missed, the follower drops from cooperative control to radar-only ACC, widens its gap gradually and alerts its driver, so the platoon separates smoothly into independent trucks. For black-box logging, each entry stores the hash of the one before it and is digitally signed, so any later edit is noticeable.

Platoon modes and maneuvers: To keep things predictable, the platoon is always in exactly one mode, and moving between modes follows fixed rules (Figure 2). A lane change, for example, is never done on a single truck's own decision. The lead truck sends a request, every truck checks its side radar and replies, the trucks behind the lane-changing one open the gap to about 25 m, and only after every acknowledgement does the truck move over. If even one acknowledgement is missing, the maneuver is cancelled and the platoon stays in its lane.

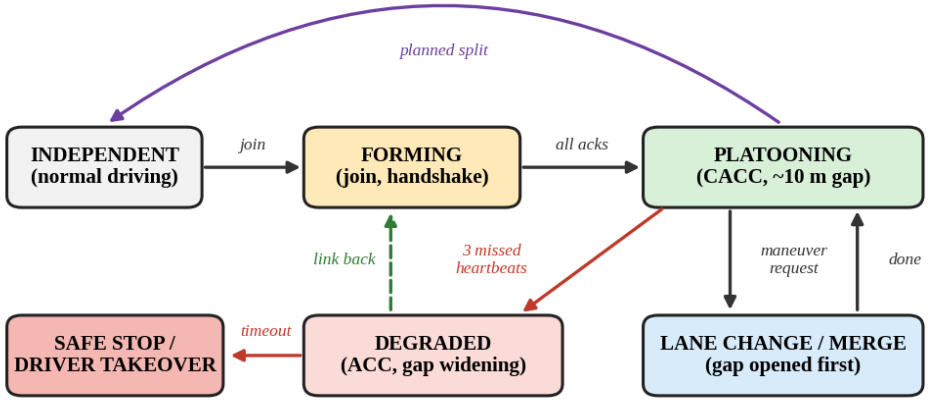


Figure 2: Platoon operating modes and transitions

Requirements:

Functional requirements	Non-functional requirements
• Collect and share radar, wheel-speed, brake and torque data in real time. • Hold the target gap with string-stable control. • Form, join and split platoons on request. • Perform coordinated lane changes and merges. • Detect communication loss and trigger safe separation. • Write signed, tamper-evident logs on every truck. • Alert drivers and show platoon status.	• Timing: control-loop sync target of 2 ms, with latency monitored. • Reliability: no single point of failure, graceful degradation. • Safety: fail-safe defaults, zero rear-end collisions as the aim. • Security: authenticated, encrypted messages. • Scalability: works for 2 to 5 trucks, extendable. • Maintainability: modular, over-the-air updates, regression tests. • Usability: simple, clear driver alerts.

Software engineering principles in the design:

Principle	How it is applied
Modularity	Each module has its own interface, so the logger or controller can be changed without touching the rest.
Scalability	The message design and maneuver logic work for any platoon size within the limit, with no truck being a bottleneck.
Maintainability	Clean layers, version-controlled code, automated tests and over-the-air updates.
Reliability	Heartbeats, redundant sensors and predictive fall-back keep working when a signal drops.
Usability	Drivers see a small number of clear states and alerts instead of raw data.
Security	Signed and encrypted V2V messages, authenticated joins, hash-chained logs.

5. Justify the Chosen Methodology/Framework

Chosen model: Agile (Scrum) with safety gates: We would build this software in 2-week sprints. Each sprint produces something we can test in a simulator, and a feature only counts as "done" after it passes a safety review and simulation tests.

Why it suits this system: Many things only become clear when we try them, such as real message delays or how the controller behaves on a slope, so short sprints let us find these early instead of at the end. The backlog can be reprioritised when tests show something is harder than expected. Waterfall is too rigid here because a timing problem found late would mean redoing a lot. The Spiral model is good for risk but heavy on paperwork, so we only borrow its short risk review in every sprint.

How it supports development and maintenance: Continuous integration runs the simulator tests on every change, so old features do not break when new ones are added. Frequent increments give regulators and the fleet operator something to review early, and the same sprint cycle is reused for updates after release. A possible plan:

Sprint	Goal
1	Requirements, hazard analysis and simulator set-up.
2	Telemetry acquisition, time sync and V2V messaging.
3	Platoon controller and string-stability tests.
4	Maneuver manager and fault handling (dropout and split).
5	Black-box logger, dashboard, integration and test-track trials.

6. Discuss Expected Outcomes and Limitations

Expected benefits: Trucks running close together should use noticeably less fuel (published trials suggest roughly 5 to 10 percent, which we would confirm in our own tests). Safer braking (the aim is no rear-end collisions even in sudden lead-truck stops), a smaller highway footprint, and continued operation if cellular or satellite coverage drops, since the trucks use their own V2V link.

How it addresses the challenges: Timing is handled by timestamps and prediction, the slinky effect by string-stable control, link failures by heartbeats and fallback, cut-ins by radar monitoring and a gap-widening reaction, and trust and liability by the shared, tamper-evident log.

Risks, implementation challenges and limitations: A real 2 ms end-to-end delay over a wireless link is very hard to guarantee, so we treat it as a target for the control loop and rely on prediction to cover small delays. The main risks and how we plan to handle them are listed below. Other limits: testing needs a lot of simulation and track time, rules differ between regions, trucks from different makers need extra integration, and the system assumes a driver on board and small highway convoys only.

Risk	Possible impact	Mitigation
Radio delay or jitter above the target	Gap errors, uncomfortable or unsafe braking.	Timestamps, local prediction, larger safe gap if delay is detected.
Communication dropout	Followers lose the lead truck’s data.	Heartbeat watchdog, fallback to radar-only ACC, driver alert.
Sensor fault or bad weather	Wrong distance or speed readings.	Cross-check radar with wheel speed, redundant sensors, wider gap.
Cyber attack or fake messages	False braking or false joins.	Signed and encrypted messages, authenticated joins.

Future enhancements: Larger platoons and platoons of platoons, weather-aware gap tuning, learning-based prediction of braking, use of HD maps and traffic data, a hybrid 5G and satellite fallback, and moving towards driver-out operation as rules allow.